

VPN-LAN Service Access — Architecture, Schemes & Diagrams

Revision: 1 **Last modified:** 2026-07-01T16:30:58Z **Status:**

Design reference — architecture, topology & data-flow diagrams for the VPN-LAN service-access feature (Phase 9 docs of [PLAN.md](#) §5) **Authority:** Inherits constitution/Constitution.md per §11.4.35. Companion to [PLAN.md](#) (§2 routing map, §3 bridge contract, §4 SSRF), [reflector_design.md](#) (Phase 5 discovery), [miracast_verdict.md](#) (Phase 8 verdict) and the operator setup guide [../guides/vpn_lan_bridge_setup.md](#). This document is the "schemes, diagrams, graphs" deliverable — it does not restate the setup steps (those live in the guide) or the per-phase task breakdown (that lives in [PLAN.md](#)); it draws the pictures those documents describe in prose. **Feature workstream:** feature/vpn-aware-dynamic-routing (§11.4.167)

0. How to read this document

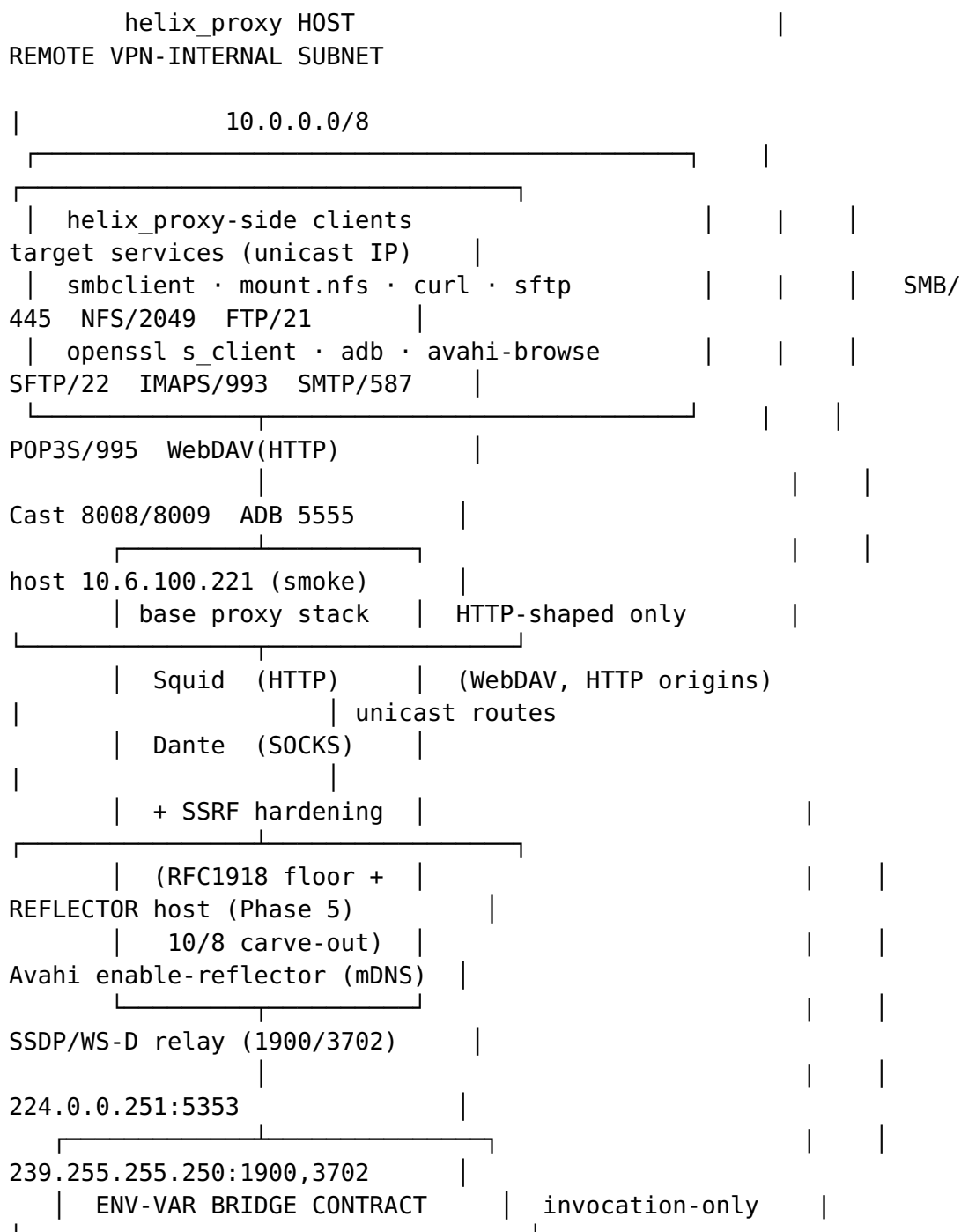
Every claim here is either grounded in the recon-derived FACTs recorded in [PLAN.md](#) §2 / §11 and the cited standards in [reflector_design.md](#) / [miracast_verdict.md](#), or explicitly marked **INFERENCE** (§11.4.6). Each diagram is drawn **twice** where the operator asked for both forms — once as an **ASCII** rendering (human-readable in the PDF export directly) and once as a **mermaid** source block (machine-renderable). Both carry the same content; neither is a raw-markup leak — the ASCII block *is* the diagram, and the mermaid block is its renderable twin (§11.4.168).

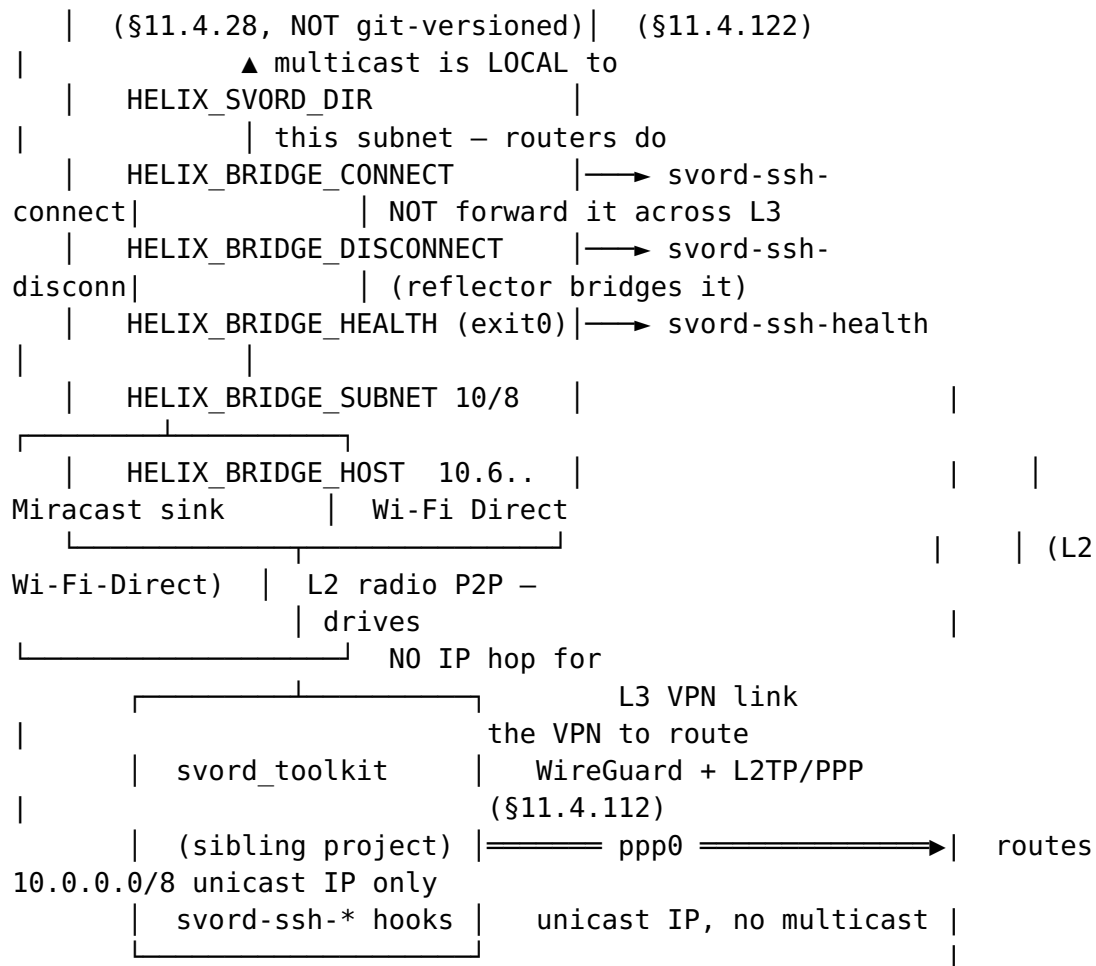
The single fact that drives every decision below: **the VPN is L3-routed** (WireGuard + L2TP/PPP over ppp0; reachable subnet 10.0.0.0/8; sword host 10.6.100.221). A Layer-3 VPN routes unicast IP packets between reachable subnets — nothing more. That one property sorts every in-scope protocol into exactly one of four buckets (§2).

1. System topology — helix_proxy ↔ sword bridge ↔ 10/8 remote subnet

The end-to-end picture: a helix_proxy-side client, the base proxy stack (Squid HTTP proxy + Dante SOCKS with the SSRF hardening), the **env-var bridge contract** that decouples helix_proxy from the sibling sword_toolkit project (§11.4.28), the L3 VPN link, and the remote 10.0.0.0/8 subnet where the target services live.

1.1 ASCII topology





Reading the picture: helix_proxy never touches sword_toolkit internals — it invokes only the four hook commands the contract names (CONNECT / DISCONNECT / HEALTH + the two data vars SUBNET / HOST), so the whole VPN mechanism is swappable by editing one .env (\$11.4.28). Everything to the right of ppp0 is reached by **unicast IP routing**; multicast (the reflector's job) is confined to the remote subnet by IP scoping rules; and Miracast's Wi-Fi-Direct radio group sits below IP entirely, where the VPN has no reach.

1.2 Mermaid topology (renderable twin)

flowchart LR

```

    subgraph HOST["helix_proxy host"]
        CL["clients: smbclient / nfs / curl /\nsftp / openssl / adb / avahi-browse"]
        PX["base proxy stack:\nSquid (HTTP) + Dante (SOCKS)\n+ SSRF hardening"]
        BR["ENV-VAR BRIDGE CONTRACT\n($11.4.28, gitignored .env)\n\nHELIX_SWORD_DIR / BRIDGE_*"]
        SV["sword_toolkit\nsword-ssh-connect/disconnect/health\n(invocation-only $11.4.122)"]
        CL --> PX
        PX --> BR
        BR --> SV
    end

```

```

end
SV ==>|"L3 VPN: WireGuard + L2TP/PPP (ppp0)\nunicast IP only,
no multicast"| RS
subgraph RS["remote subnet 10.0.0.0/8"]
SVC["unicast services\nSMB NFS FTP SFTP IMAPS\nSMTP POP3S
WebDAV Cast ADB"]
RF["REFLECTOR host (Phase 5)\nAvahi enable-reflector
(mDNS)\nSSDP/WS-D relay 1900/3702"]
MC["Miracast sink\nWi-Fi Direct L2 radio P2P\nno routable
IP hop ($11.4.112)"]
RF -->|"bridges multicast\nlocal to this subnet"| SVC
end
CL -. "L2 radio proximity -\nVPN cannot reach" .-> MC

```

2. Per-protocol routing decision table

Every in-scope protocol maps to exactly one of four **primitives**, decided by its traffic class (FACT, [PLAN.md](#) §2). This is the master decision table — the "WHY" column is the load-bearing reason, not decoration.

Protocol / service	Ports	Traffic class	Primitive	WHY (the deciding factor)
SMB / CIFS	445	Unicast TCP	L3-ROUTE	A mount of unicast IP service; SOCKS5 is the wrong primitive for mounts. Over ppp0
NMB / NetBIOS name res.	137-138	Unicast UDP (fallback)	L3-ROUTE	Broadcast multicast is not routed across L3 the shared 10.x IP (unlike rather than NetBIOS)
NFS	2049 (+ aux)	Unicast TCP/UDP	L3-ROUTE	Same as a routed not a protocol one. Needs routing, not SOCKS5.
FTP control	21	Unicast TCP	L3-ROUTE	Control of is a plain TCP connection

Protocol / service	Ports	Traffic class	Primitive	WHY (the deciding factor)
FTP/FTPS data (passive)	pinned PASV range	Unicast TCP	L3-ROUTE (pinned range)	Passive data transfer uses a separate port the server advertises in the server's pinned port range. FTPS encryption of the control channel requires a proxy ALIAS which cannot receive PASV — not a proxy (not proxied) the cleanest.
SFTP	22	Unicast TCP (over SSH)	L3-ROUTE	Single connection SSH — recommended modern proxy WebDAV
WebDAV	HTTP(S)	HTTP-shaped	SQUID-PROXY (existing)	HTTP (RFC 4918) — already works through the existing (PROPFIND) no new components
IMAP / IMAPS	993	Unicast TCP	L3-ROUTE	Implicit-TLS mail retrieval a routed connection 8314 preferred over STARTTLS
SMTP submission	587 / 465	Unicast TCP	L3-ROUTE (auth only)	Route authentication submission VPN client never exposed anonymously CONNEC (open-reliable guard, \$4

Protocol / service	Ports	Traffic class	Primitive	WHY (the deciding factor)
POP3 / POP3S	995	Unicast TCP	L3-ROUTE	Implicit-7 retrieval, unicast.
Google Cast control	8008 / 8009 (TLS)	Unicast TCP	L3-ROUTE	eureka_in HTTP (80 CASTV2 TLS) are — they re once the device's is known
DIAL control	HTTP	Unicast TCP	L3-ROUTE	DIAL con plain uni HTTP to advertise LOCATION.
ADB (connect/debug)	5555	Unicast TCP	L3-ROUTE	adb connect 10.x:5555 the route — no pro central a server m
mDNS / DNS-SD	224.0.0.251:5353	Multicast discovery	REMOTE-REFLECTOR	Link-local (RFC 577 Network Block); T routers M NOT forw Nothing VPN to r reflect on remote s
SSDP / UPnP	239.255.255.250:1900	Multicast discovery	REMOTE-REFLECTOR	Administr scoped g (RFC 236 TTL, no multicast routing s across th tunnel → on the re subnet.
WS-Discovery	239.255.255.250:3702	Multicast discovery	REMOTE-REFLECTOR	Same ad scoped g SSDP; SC over-UDP discovery

Protocol / service	Ports	Traffic class	Primitive	WHY (the deciding factor)
Chromecast/DIAL discovery	5353 / 1900	Multicast discovery	REMOTE-REFLECTOR	subnet-local reflect. Cast discovery (_googlecast.mDNS) + (M-SEARCH) multicast discovery reflects – control the routes (routed above).
ADB flash (fastboot)	USB	USB-level (not IP)	usbip (USB-over-IP)	fastboot : USB protocol not a routing service; control via usbip remote host the device attached network is honest bound (§11.4.13)
Miracast	— (Wi-Fi Direct)	L2 radio P2P	STRUCTURALLY-IMPOSSIBLE	Wi-Fi Direct forms a L2 radio group between devices in RF proximity — no routing hop exists L3 VPN tunnel (§11.4.11) Google Chromecast DIAL instances

One-line rule of thumb (FACT, [PLAN.md](#) §2): *unicast services route; HTTP-shaped services already work through Squid; multicast discovery needs a remote-side reflector; Wi-Fi-Direct / L2 is out of scope by physics.*

2.1 Primitive-selection decision tree (mermaid)

flowchart TD

```
A["a service on the 10/8 remote subnet"] --> B["Is it an IP service\n(has a routable address)?"]
```

```

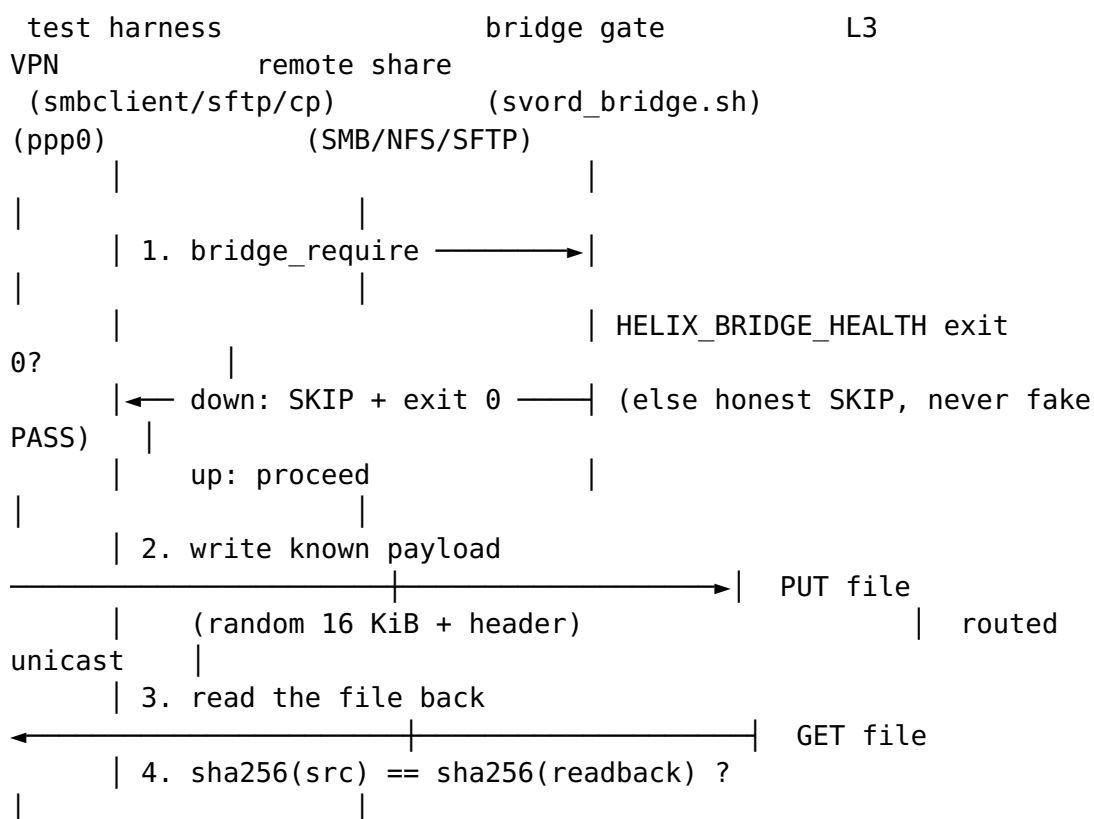
    B -- No --> M{"Wi-Fi Direct /\nL2 radio?"}
    M -- Yes --> MI["STRUCTURALLY-IMPOSSIBLE\nMiracast
($11.4.112)\nuse Google Cast instead"]
    M -- "No (USB, e.g. fastboot)" --> UB["usbip (USB-over-IP)
\noperator-gated $11.4.133"]
    B -- Yes --> C{"Is it multicast\ndiscovery?"}
    C -- Yes --> RF["REMOTE-REFLECTOR\nmDNS/SSDP/WS-D\n(discovery
only;\ncontrol then routes)"]
    C -- No --> D{"Is it HTTP-shaped?"}
    D -- Yes --> SQ["SQUID-PROXY (existing)\nWebDAV, HTTP
origins"]
    D -- No --> RT["L3-ROUTE over ppp0\nSMB NFS FTP SFTP\nIMAPS
SMTP POP3S\nCast/DIAL control ADB"]

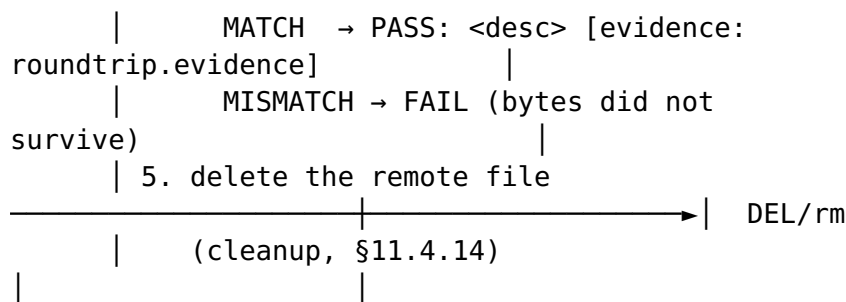
```

3. Data-flow diagrams

3.1 File round-trip (SMB / NFS / SFTP — byte-integrity path)

The file-share and file-transfer tests all follow the same **write → read-back → sha256** integrity loop (FACT — tests/vpn_lan/smb_nfs_roundtrip.sh, tests/vpn_lan/ftp_sftp_webdav.sh). A PASS is emitted **only** when the source sha256 equals the read-back sha256 (integrity: MATCH); anything else SKIPS or FAILs. The bridge gate runs first — bridge down = honest SKIP:network_unreachable_external, never a fake PASS.





▼
▼

evidence: qa-results/vpn_lan/phase{2,3}/<UTC-ts>/{smb,nfs,sftp}/
roundtrip.evidence
(src_sha256, readback_sha256, integrity: MATCH/
MISMATCH)

sequenceDiagram

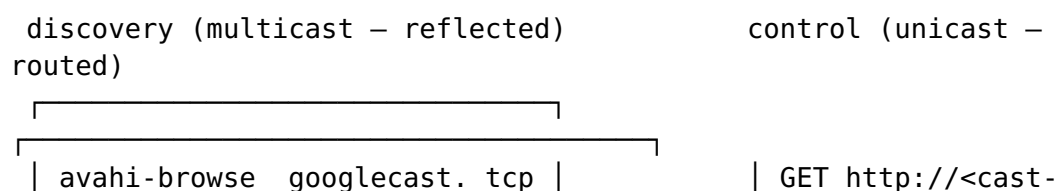
```

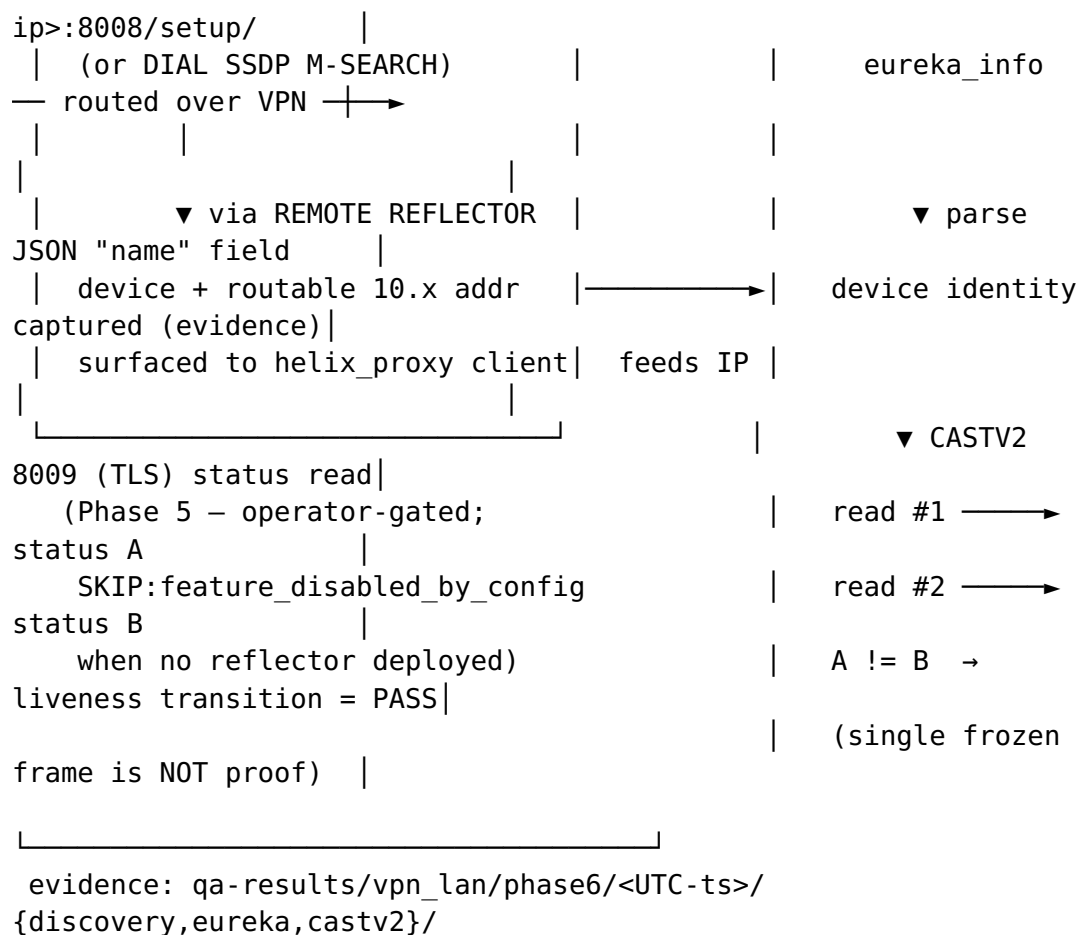
    participant T as test harness
    participant G as bridge gate (svord_bridge.sh)
    participant V as L3 VPN (ppp0)
    participant S as remote share
    T->>G: bridge_require
    alt bridge down
        G-->>T: SKIP:network_unreachable_external (exit 0)
    else bridge up
        G-->>T: proceed
        T->>V: write known payload (16 KiB + header)
        V->>S: PUT file (routed unicast)
        T->>V: read file back
        V->>S: GET file
        S-->>T: bytes
        T->>T: sha256(src) == sha256(readback)?
        Note over T: MATCH -> PASS (evidence:
roundtrip.evidence)<br/>MISMATCH -> FAIL
        T->>S: delete remote file (cleanup §11.4.14)
    end

```

3.2 Cast control flow (Google Cast / DIAL)

Cast splits into **discovery** (multicast — needs the Phase-5 reflector) and **control** (unicast — routes directly). The test (FACT — tests/vpn_lan/chromecast_dial.sh) reads the device-identity JSON from eureka_info (8008) and observes a **status transition** on CASTV2 (8009 TLS) — a transition, not a single frozen frame, per §11.4.107.





sequenceDiagram

```

sequenceDiagram
    participant C as helix_proxy client
    participant R as remote reflector (Phase 5)
    participant D as Cast device (10.x)
    Note over C,R: DISCOVERY (multicast – reflected)
    C->>R: avahi-browse _googlecast._tcp / DIAL M-SEARCH
    R-->>C: device + routable 10.x address
    Note over C,D: CONTROL (unicast – routed over VPN)
    C->>D: GET :8008/setup/eureka_info
    D-->>C: JSON name (device identity = evidence)
    C->>D: CASTV2 :8009 TLS status read #1
    D-->>C: status A
    C->>D: CASTV2 :8009 TLS status read #2
    D-->>C: status B
    Note over C: A != B -> liveness transition = PASS
    Note over C: (single frozen frame is NOT proof)
    Note over C: evidence: qa-results/vpn_lan/phase6/<UTC-ts>/
    Note over C: {discovery,eureka,castv2}/
  
```

4. SSRF-allowlist trust-boundary diagram

Widening egress to the VPN subnet reopens SSRF surface, so the carve-out is added **without** removing the existing hardening (FACT, [PLAN.md](#) §4). The Dante ACL is **first-match, top-down**: the

over STARTTLS (RFC 8314)
the carve-out.

guards stay GREEN after

4.2 Trust-boundary decision (mermaid)

flowchart TD

```
REQ["egress request target"] --> R1{"in  
HELIX_BRIDGE_SUBNET\n(e.g. 10.0.0.0/8)?"}  
R1 -- Yes --> ALLOW["ROUTE over ppp0\n(TRUSTED – the narrow  
carve-out)"]  
R1 -- No --> R2{"RFC1918 / link-local /\nloopback /\nmetadata?"}  
R2 -- Yes --> DENY["BLOCK\nDante block(N) / TCP_DENIED\n(the  
FLOOR – kept intact)"]  
R2 -- No --> PUB["ALLOW public internet\n(unchanged base  
policy)"]  
ALLOW --> MUT["paired §1.1 mutation:\nan out-of-allowlist  
RFC1918 target\nMUST still deny (teeth)"]  
DENY --> MUT
```

The invariant this diagram encodes (FACT, [PLAN.md §4](#)): the carve-out is the *only* new hole, it is exactly one subnet wide, it sits above the deny in first-match order, and every widening ships with a mutation proving an out-of-allowlist target still denies — so the allowlist can never silently drift permissive. The email open-relay guard is a second, orthogonal boundary on the same egress path: authenticated submission is routed, anonymous CONNECT-to-:25 is never exposed, so the SSRF carve-out cannot turn helix_proxy into an anonymizing spam conduit.

5. Honest boundaries encoded in the architecture (§11.4.6)

- **Multicast never crosses L3.** The reflector is not an optimization — it is *required* because link-local (224.0.0.251, RFC 5771) and administratively-scoped (239.255.255.250, RFC 2365) groups are subnet-local by standard mandate. See [reflector_design.md §2](#) for the full routing argument.
- **Miracast has no IP hop.** Its Wi-Fi-Direct L2 radio group sits below IP; the VPN cannot manufacture a radio association. Classified Won't-fix: structurally-impossible (§11.4.112) — see [miracast_verdict.md](#). The routable alternative is Google Cast/DIAL (§3.2).
- **Network fastboot is USB-bound.** fastboot is a USB protocol; the routable path is usbip. Flashing a real device is operator-gated (§11.4.133).
- **Bridge-down ⇒ honest SKIP.** Every diagram's data flow begins with the bridge gate; when the VPN is down the live

paths **SKIP** (`network_unreachable_external`, `exit 2`), never fake-PASS. The architecture is provable against local stubs for the security-critical logic (SSRF teeth, the doctor), and honestly SKIPs the live protocol paths until the operator supplies the sword connection ([PLAN.md](#) §6 / §8).

- **INFERENCE items** — the specific SSDP-relay binary (§4.1 of [reflector_design.md](#)) is a deployment-time choice, not asserted here. The diagrams show the *requirement* (an SSDP-aware relay that rewrites LOCATION to a routable 10.x), not a pinned tool.
-

Sources

- [PLAN.md](#) — §2 routing map (the four traffic classes → four primitives), §3 env-var bridge contract, §4 SSRF reconciliation, §5 per-phase protocol coverage, §11 deep-research citations.
- [reflector_design.md](#) — the multicast-does-not-cross-L3 argument (RFC 6762/6763/5771/2365, Avahi enable-reflector) drawn in §1/§2/§3.2 here.
- [miracast_verdict.md](#) — the §11.4.112 structural-impossibility verdict (Wi-Fi Alliance / Wi-Fi Direct) behind the Miracast row of §2 and §5.
- [../..guides/vpn_lan_bridge_setup.md](#) — the operator setup guide (the .env contract, doctor verdicts) these diagrams illustrate.
- Test scripts (the data-flow FACTs of §3): `tests/vpn_lan/smb_nfs_roundtrip.sh`, `tests/vpn_lan/ftp_sftp_webdav.sh`, `tests/vpn_lan/chromecast_dial.sh`, `tests/vpn_lan/discovery_reflect.sh`, `tests/vpn_lan/adb_over_vpn.sh`, `tests/vpn_lan/email_roundtrip.sh`, `tests/lib/sword_bridge.sh`.